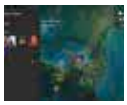

Quick Resume to be even faster
The May update for Xbox Series S/X will improve the effeciency of the "Quick Resume" feature. http://dgit.in/jun21-35

Whatsapp won't limit functionality
The company decided against limiting user functionality if they didn't accept its new privacy policy. http://dgit.in/jun21-36

would be to place the mouse on the button in a fresh game tab, and have the following code execute

```
print(pyautogui.position())
```

The output for this will give us the exact coordinates on the screen at which the play button resides.

In our PC, based on our screen resolution, we saw that they are present at (958,556).

In a similar way, we need find the location of the dino and again, in our PC, it is present at (207,610). We record these two locations into variables as we will need them from time to time.

```
PlayButton=(958,556)
dinoImage=(207, 610)
```

To re-spawn the game every time we hit an obstacle, we write a very tiny function that looks like the image here where we simply call the click function of the pyautogui module and pass the PlayButton as a parameter, and just to ensure that we have clicked the button, we allow a print on the console (this is entirely optional).

```
def restartGame():
    pyautogui.click(PlayButton)
    print("play button clicked")
```

With these two out of the way, let's start building the logic. First, we would need some time to allow ourselves to switch back into the chrome tab and for that purpose, we need some timed delay. We sleep for any arbitrary number of seconds. We've gone with 3 in our case as it works fine for us.

```
time.sleep(3)
```

We call the function to restart the game followed by a while loop, which is to ensure a continued time period within which we shall automate our hitting the space button at the necessary scenarios.

The pressSpace function is relatively very easy to understand where we use the pyautogui's keyDown function and pass on the parameter of the string 'space' which will automate hitting the space bar.

```
def pressSpace():
    pyautogui.keyDown('space')
```

At this point, we talk of another function within which we figure out if there is any obstacle present right in front of the dino. To do that, we select a box of dimensions 90 by 15 pixels starting right at the beginning of the dino on the screen. We use the grab function of the ImageGrab library to take a screenshot of the same and in turn use the grayscale function of the ImageOps library to give us the converted from RGB into grayscale value. The black and white values will further help us in our analysis of any given obstacle or otherwise. The next thing that we do is to get the individual intensity of every pixel in this grayscale image and store them into an array. The reason for using an array here is to get the elements stored in an orderly manner which also helps us establish any pattern (if necessary) to get help on the analysis of the structure that was image grabbed earlier.

With this hand, all we need is the sum of the values in the array and here we need a bit of experimentation. Back to the game in the browser, we run this game for a while and check the values that come up in the console, returned by the image_grab function. For us, the values that worked best were 1605. Any value less than 1605 meant that there were lower intensities in the pixels, meaning, an essentially darker color all throughout the image. The moment we had any obstacle on the way, we saw that the sum of the intensity values of the pixels in that screen-grabbed image shot up from the threshold that we considered at 1605, enabling us to understand that there were obstacles.

At this point, we have an idea of whether or not, there is an obstacle present here and all we need to do is jump accordingly. This can be done easily as is shown in the image below where we check for the value returned by the image grab function and if it doesn't match the threshold, we simply call the pressSpace function for the dino to jump. This is all there is to it. However, if you still notice, there is a certain timed sleep of 0.1 seconds that has been added to pause the flow of control, only momentarily before continuing. It was seen that with the delay not present, the image_Grab function would be immediately called and with its returned value (which would still hold a part of the obstacle- held back from the previous obstacle), it would make our dino jump again even when there was no new obstacle, and that is something we don't want. Hence, the delay was added.

```
26  while True:
27      if (image_Grab()!=1605):
28          pressSpace()
29          time.sleep(0.1)
```

```
def image_Grab():
    box=(dinoImage[0]+55, dinoImage[1], dinoImage[0]+145, dinoImage[1]+15)
    image=ImageGrab.grab(box)
    grayImage=ImageOps.grayscale(image)
    a=array(grayImage.getcolors())
    return a.sum()
```

With this, we come to the end of this fairly simple project where we automated a mouse click and a keyboard key press. Added to that, we managed to take a screenshot, got the coordinates of the mouse pointer on any particular area on the screen and ultimately managed to automate the dino game of Google Chrome with the bare minimum necessity.